CYBER SECURITY INTERNSHIP

TREINETIC

Prepared By: W. Saneth Fernando
Date: 23/3/2026

Target Application – DVWA (Damn Vulnerable Web Application)
Testing Tools – Burp Suite Community Edition, Browser Dev Tools

1. Introduction

Web application security testing is an area I have been actively working on as part of my cyber security studies. This assessment covers the identification, exploitation, and analysis of common web vulnerabilities across a deliberately vulnerable application.

The target application used was DVWA, deployed locally using Docker. DVWA is a well-known platform used in security testing practice and covers the key vulnerability categories relevant to this assessment.

The five vulnerabilities tested were SQL Injection, Cross-Site Scripting (XSS), Insecure Direct Object Reference (IDOR), Authentication Issues, and Security Misconfiguration. For each one, the vulnerability is explained, the testing steps are documented with evidence, and a fix is recommended.

All testing was done by my personal local machine without having any relation to real world data.

2. Setup

The steps below describe how I set up the testing environment before starting any vulnerability testing.

- Installed Docker Desktop on Windows and confirmed it was running.
- Ran DVWA using the command “docker run --rm -it -p 80:80 vulnerables/web-dvwa”.
- Opened “http://localhost/setup.php” in the browser and clicked “Create/Reset Database”.
- Logged in with the default credentials (username: admin, password: password).
- Set the security level to Low under DVWA Security so vulnerabilities could be clearly demonstrated.
- Installed Burp Suite Community Edition and used the built-in Burp browser to route all traffic through the proxy.

The security level was set to ‘low’ so that the vulnerabilities are visible and exploitable without the application's own filters getting in the way. In a real production application, these protections would typically not exist at all, which is exactly why identifying and fixing them is important.

3. Vulnerability Identification and Testing

The following sections cover each vulnerability I found during testing. Each one includes an explanation of the issue, the steps I took, screenshots as evidence, and Burp Suite captures where relevant.

3.1 SQL Injection

SQL Injection is a vulnerability where user input gets inserted directly into a SQL database query without any filtering. This lets an attacker change the logic of the query, pull out data they should not have access to, or bypass login checks entirely.

The path that it is in is the 'SQL Injection' section in DVWA. (http://localhost/vulnerabilities/sqli/)

Test 1: Normal Input

I entered the value 1 to check that the form was working normally and querying the database. The attached is the baseline.

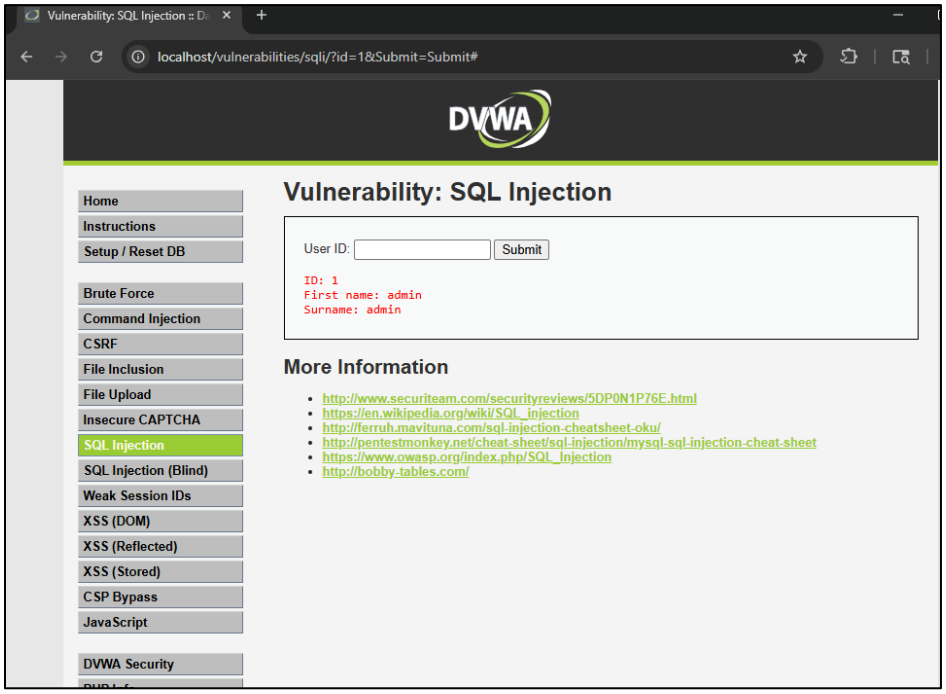


Figure 1: Normal input showing the expected user record for ID 1

Test 2: Triggering a SQL Error

I entered 1' (with a single quote) to break the SQL syntax. The application returned a raw database error message, which confirms that the input is going directly into the SQL query with no validation.

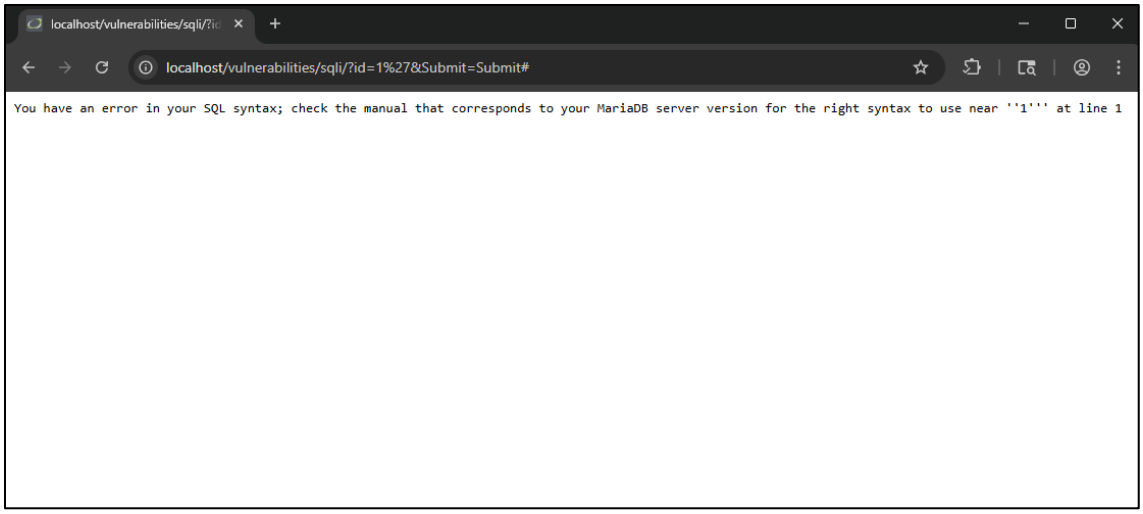


Figure 2: error caused by the single quote, confirming the vulnerability

Test 3: Burp Suite Intercept

With Burp Suite intercept turned on I submitted the form and captured the HTTP request. This shows the id parameter being sent in the URL with no protection or encoding applied.

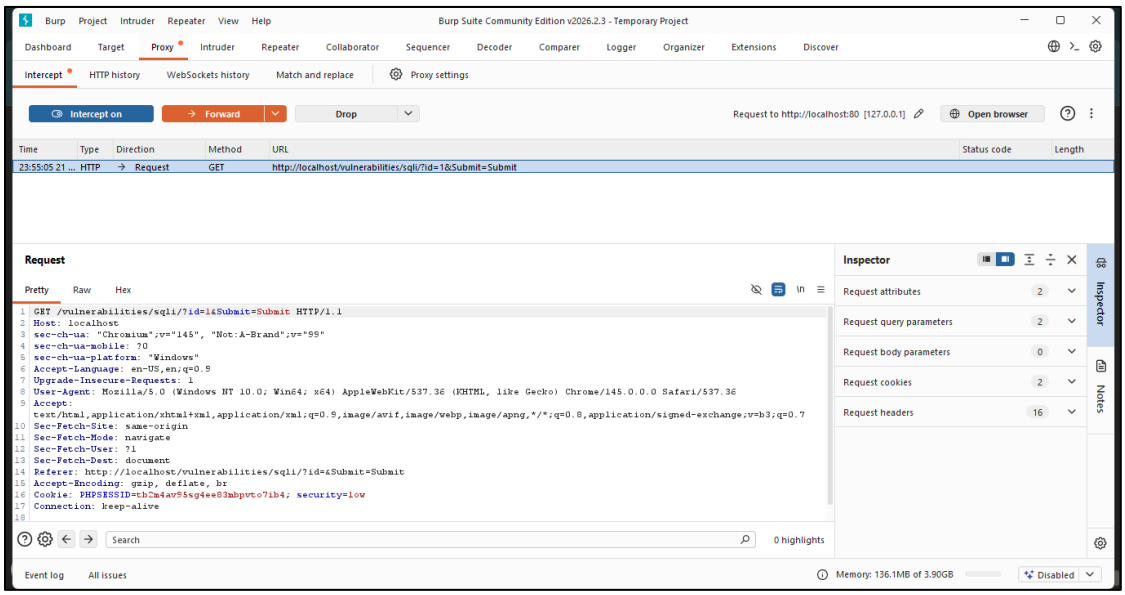


Figure 3: Burp Suite intercepting the SQL Injection request

Test 4 – Dumping all the records

I used the payload 1' OR '1'='1 which makes the WHERE condition always true. Instead of returning one user, the database returned every user record, showing that the data can be extracted without authorization.

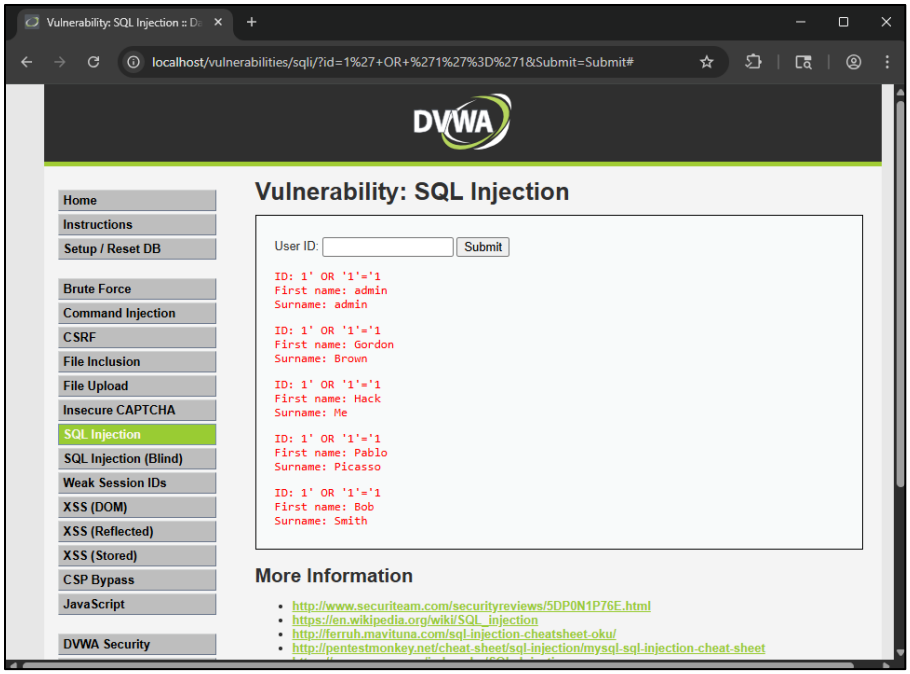


Figure 4: All user records returned by the OR-based injection payload

Test 5: UNION SELECT to extract database info

Using the payload 1' UNION SELECT user(),database()-- - I was able to pull the current database username and database name into the page output. This kind of information helps an attacker understand the backend and plan further attacks.

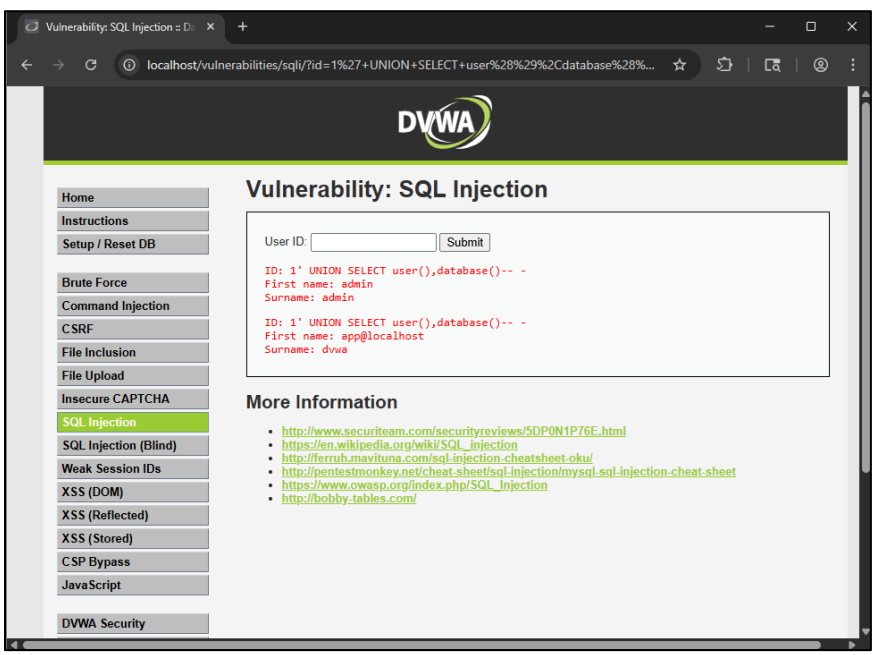


Figure 5: UNION SELECT showing the database user and database name

3.2 Cross-Site Scripting (XSS)

Reflected XSS happens when user input is reflected back in the page response without being properly encoded. If an attacker injects JavaScript, the browser will run it. This can be used to steal cookies, hijack sessions, or redirect users to malicious pages.

The path that it is in is the XSS (Reflected) section in DVWA AND the name input field at http://localhost/vulnerabilities/xss_r/ .

Test 1 – Normal Input

I entered my name to confirm the input gets reflected in the page. This sets the baseline for the test.

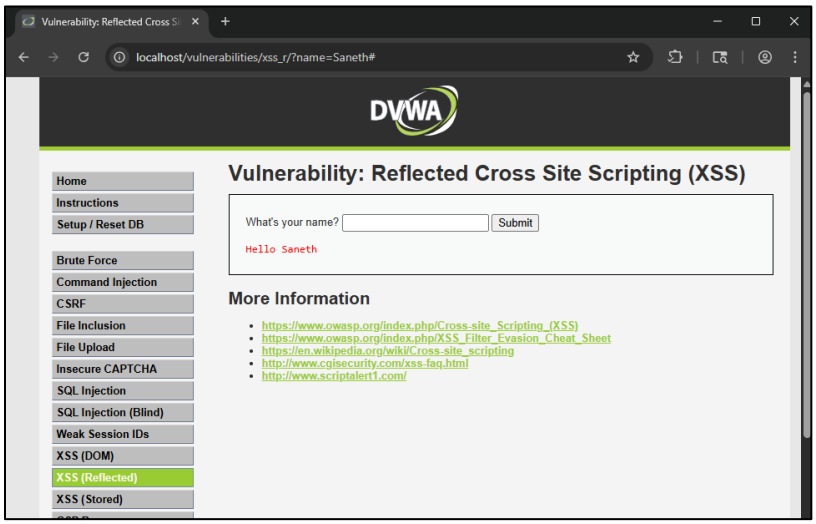


Figure 6: Normal input being reflected in the page response

Test 2: Script Injection

I entered the payload "<script>alert('XSS')</script>" in the name field. The browser executed the script and a popup appeared, confirming that the application is rendering user input as HTML without any sanitization.

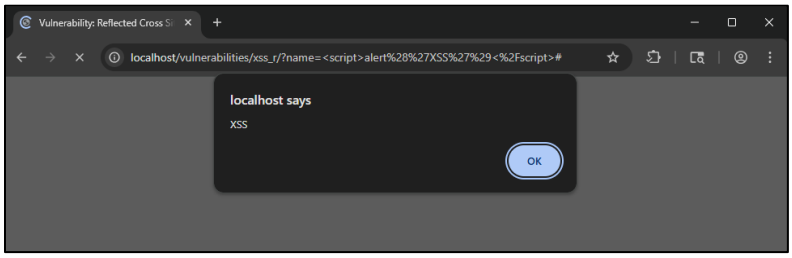


Figure 7: Alert popup confirming the XSS script executed in the browser

Test 3: Burp Suite Capture

I captured the XSS request in Burp Suite. The screenshot shows the script payload being sent as a raw URL parameter and returned in the response with no encoding applied.

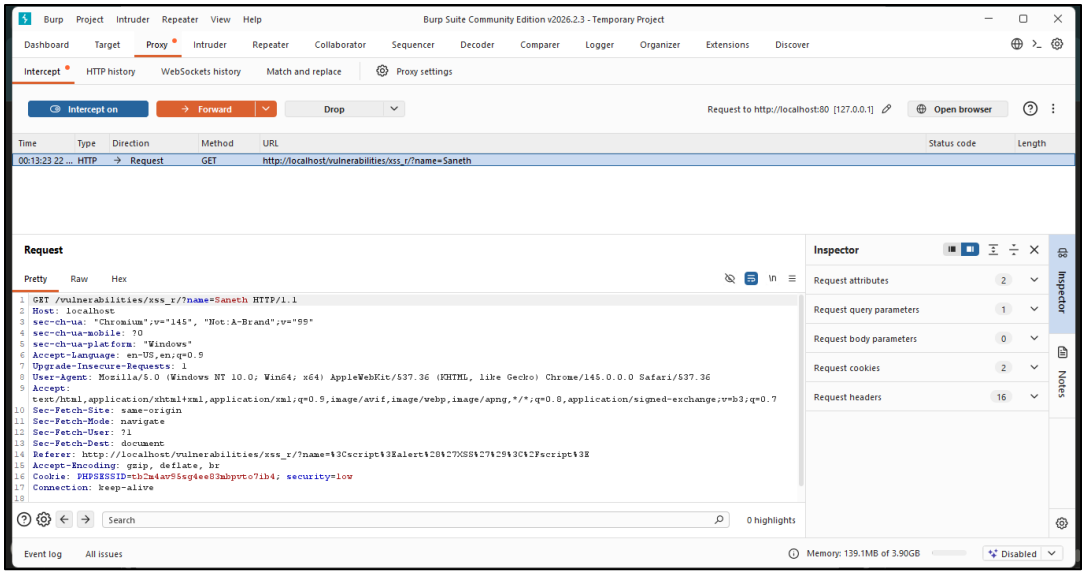


Figure 8: Burp Suite showing the XSS request with the script payload

3.3 Insecure Direct Object Reference (IDOR)

IDOR is when an application uses a user supplied value like an ID number to fetch a resource, without checking whether the user has permission to access that specific resource. An attacker can simply change the number to access someone else's data.

The path that is in the SQL Injection Page section in DVWA. By changing the id parameter in the URL, different user records are returned with no authorization check.

By changing the id value in the URL from 1 to 2 and then 3, I was able to retrieve the personal details of completely different users. The application returns the data without checking whether I should be allowed to see it. The screenshots below show three separate user records being accessed just by incrementing the ID number.

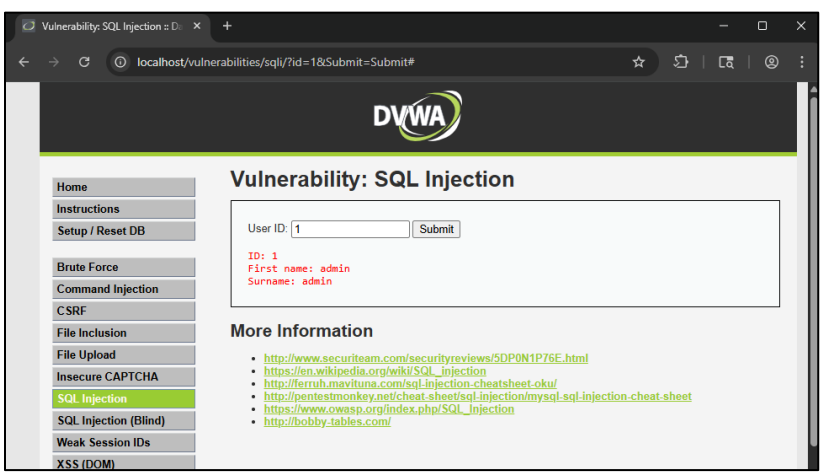


Figure 9: User record for ID 1 (admin) accessed via the id parameter in the URL

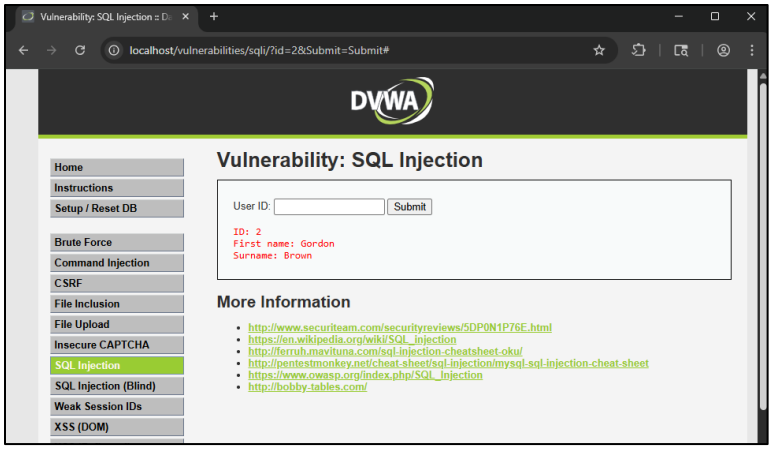


Figure 10: User record for ID 2 (Gordon Brown) returned by changing id=1 to id=2

Figure 11: User record for ID 3 (Hack Me) returned by changing id=2 to id=3, confirming no authorization check exists

3.4 Authentication Issues - No Brute Force Protection

When a login page does not limit the number of attempts, an attacker can keep trying passwords automatically until they find the right one. There is no lockout, no delay, and no CAPTCHA on this login form, which makes it straightforward to brute force.

The location for this is under the 'Brute Force' Section in DVWA - http://localhost/vulnerabilities/brute/ .

Test 1 – Failed login with no lockout

I submitted a wrong password for the admin account. The application showed an error but nothing else happened. There were no delays, no lockout, and no CAPTCHA. This means an attacker could keep trying indefinitely.

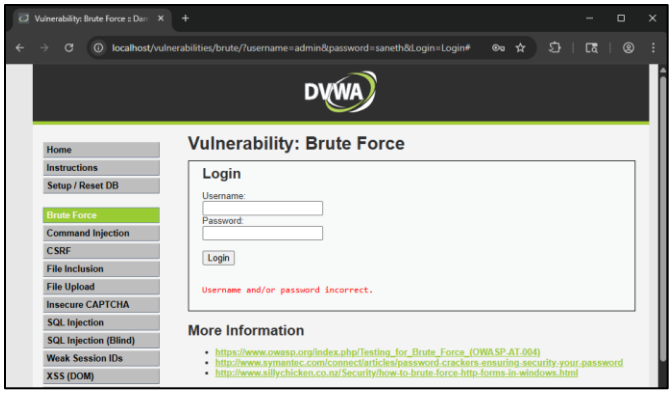


Figure 12: Failed login attempt with no rate limiting or account lockout

Test 2: Burp Suite Request Capture

I intercepted the login request in Burp Suite. The credentials are sent as plain text GET parameters in the URL, which means the request can easily be replayed or automated.

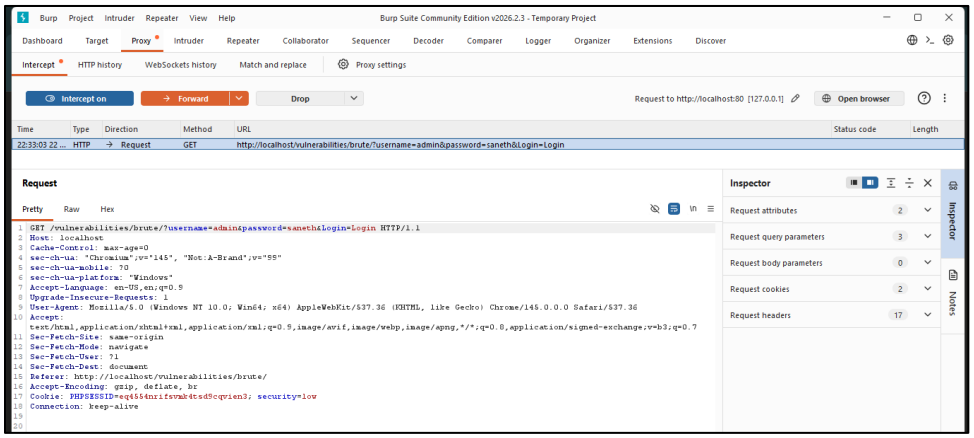


Figure 13: Burp Suite showing login credentials in the GET request parameters

Test 3: Successful login

I then logged in with the correct credentials (admin / password). There was no second factor, no CAPTCHA, and no indication the system had noticed the failed attempts before.

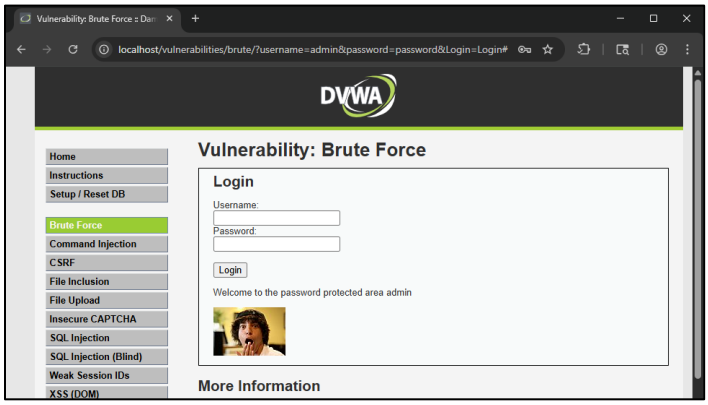


Figure 14: Successful login after brute forcing, no additional verification required

3.5 Security Misconfiguration

Security misconfiguration covers issues like missing response headers, sensitive files left accessible, or insecure default settings. These are some common things and most of the time those give attackers useful information even before they start actively exploiting anything.

Finding 1 - Missing HTTP security headers

I checked the HTTP response headers using Burp Suite and found that several important security headers were missing from every response. Each of these headers prevents a specific type of attack and they are straightforward to add.

The headers that were missing:

- X-Frame-Options: without this the page can be embedded in an iframe on another site, enabling clickjacking.
- X-Content-Type-Options: without this browsers may guess the content type, which can be exploited.
- Content-Security-Policy: without this there are no restrictions on where scripts can be loaded from.
- Strict-Transport-Security: without this the application does not enforce HTTPS.
- X-XSS-Protection: the browser level XSS filter is not turned on.

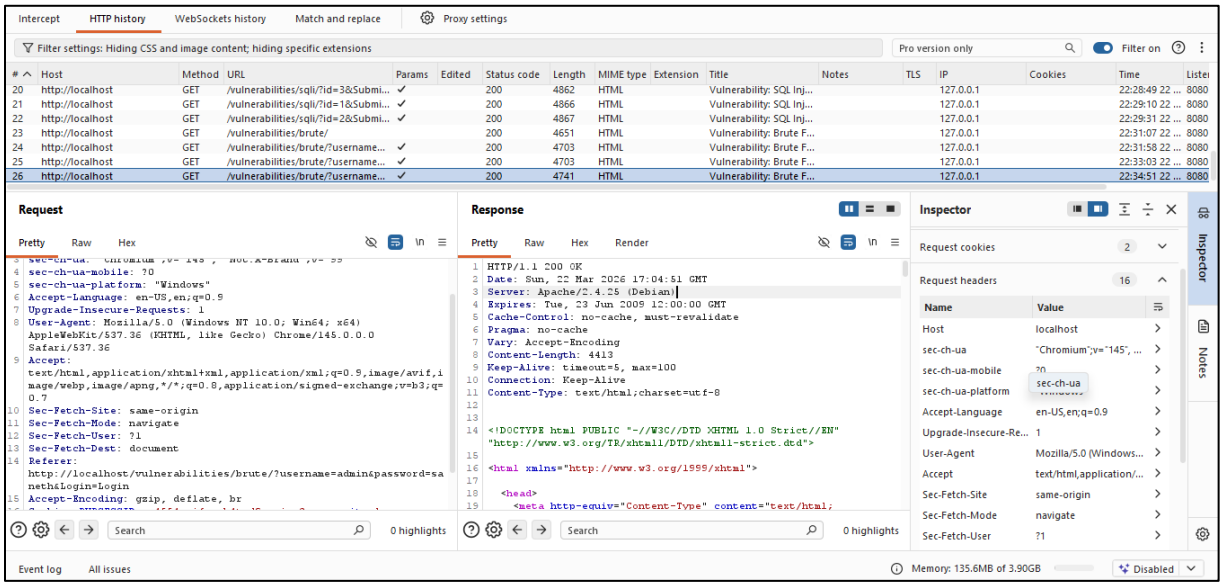
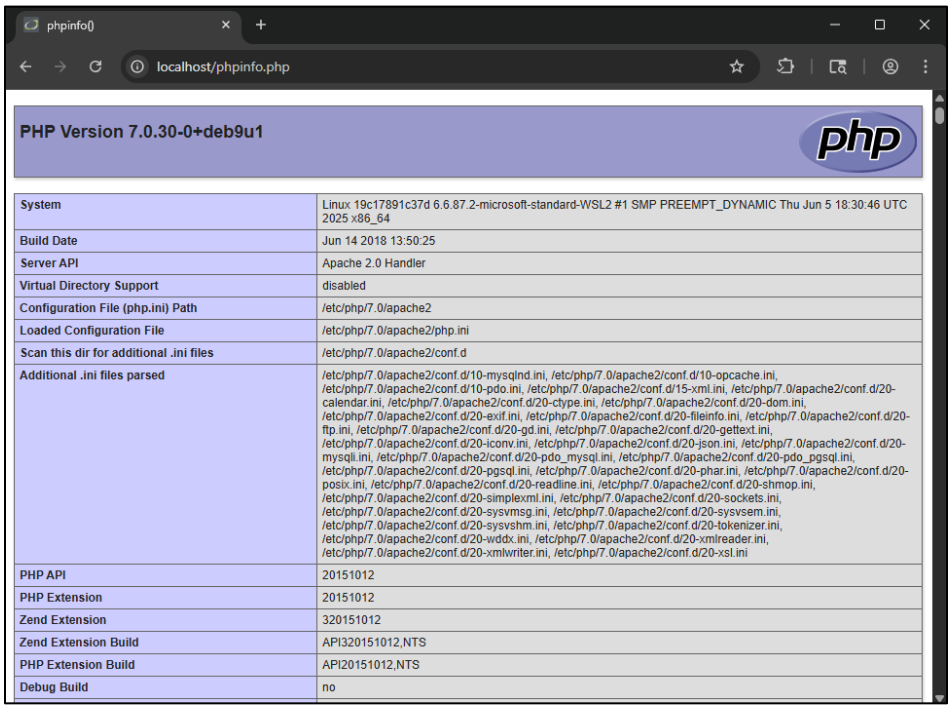


Figure 15: Burp Suite response showing the security headers are absent

Finding 2 – 'phpinfo.php' publicly accessible

The file 'phpinfo.php' was accessible at http://localhost/phpinfo.php without any login required. This page displays the full server configuration including the PHP version (7.0.30, which has been end of life since December 2019), the Apache version, all loaded modules, internal file paths, and

environment variables. This is information that should never be visible to anyone outside the server.

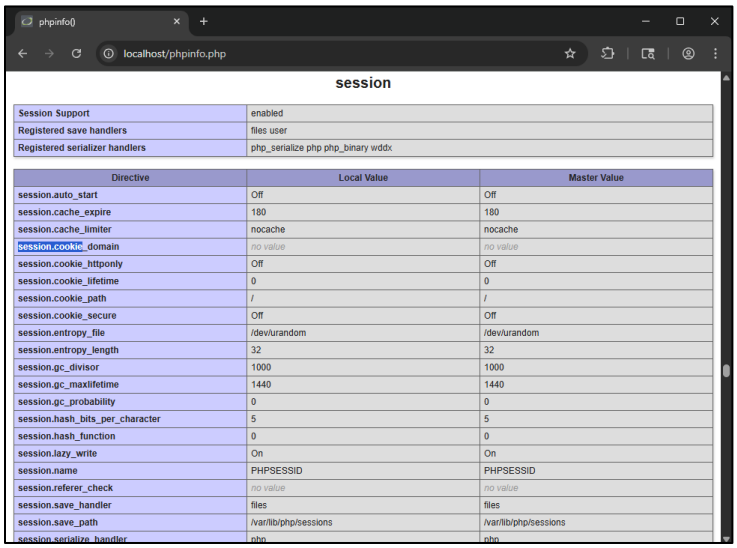


System	Linux 19c17891c37d 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025 x86_64
Build Date	Jun 14 2018 13:50:25
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.0/apache2
Loaded Configuration File	/etc/php/7.0/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.0/apache2/conf.d
Additional .ini files parsed	/etc/php/7.0/apache2/conf.d/10-mysqlnd.ini, /etc/php/7.0/apache2/conf.d/10-opcache.ini, /etc/php/7.0/apache2/conf.d/10-pdo.ini, /etc/php/7.0/apache2/conf.d/15-xml.ini, /etc/php/7.0/apache2/conf.d/20-calendar.ini, /etc/php/7.0/apache2/conf.d/20-ctype.ini, /etc/php/7.0/apache2/conf.d/20-dom.ini, /etc/php/7.0/apache2/conf.d/20-exif.ini, /etc/php/7.0/apache2/conf.d/20-fileinfo.ini, /etc/php/7.0/apache2/conf.d/20-ftp.ini, /etc/php/7.0/apache2/conf.d/20-gd.ini, /etc/php/7.0/apache2/conf.d/20-gettext.ini, /etc/php/7.0/apache2/conf.d/20-iconv.ini, /etc/php/7.0/apache2/conf.d/20-json.ini, /etc/php/7.0/apache2/conf.d/20-mysqli.ini, /etc/php/7.0/apache2/conf.d/20-pdo_mysql.ini, /etc/php/7.0/apache2/conf.d/20-pdo_pgsql.ini, /etc/php/7.0/apache2/conf.d/20-pgsql.ini, /etc/php/7.0/apache2/conf.d/20-phar.ini, /etc/php/7.0/apache2/conf.d/20-posix.ini, /etc/php/7.0/apache2/conf.d/20-readline.ini, /etc/php/7.0/apache2/conf.d/20-shmop.ini, /etc/php/7.0/apache2/conf.d/20-simplexml.ini, /etc/php/7.0/apache2/conf.d/20-sockets.ini, /etc/php/7.0/apache2/conf.d/20-sysvmsg.ini, /etc/php/7.0/apache2/conf.d/20-sysvsem.ini, /etc/php/7.0/apache2/conf.d/20-sysvshm.ini, /etc/php/7.0/apache2/conf.d/20-tokenizer.ini, /etc/php/7.0/apache2/conf.d/20-wddx.ini, /etc/php/7.0/apache2/conf.d/20-xmlreader.ini, /etc/php/7.0/apache2/conf.d/20-xmlwriter.ini, /etc/php/7.0/apache2/conf.d/20-xsl.ini
PHP API	20151012
PHP Extension	20151012
Zend Extension	320151012
Zend Extension Build	API320151012,NTS
PHP Extension Build	API20151012,NTS
Debug Build	no

Figure 16: phpinfo.php exposing PHP 7.0.30 and full server configuration details

Finding 3: Insecure session cookie settings

From the 'phpinfo' page I could see that both 'session.cookie_httponly' and 'session.cookie_secure' were set to Off. This means session cookies can be read by JavaScript (making them stealable through XSS) and are sent over plain HTTP connections without encryption.



Session Support	enabled
Registered save handlers	files user
Registered serializer handlers	php_serialize php php_binary wddx

Directive	Local Value	Master Value
session.auto_start	Off	Off
session.cache_expire	180	180
session.cache_limiter	nocache	nocache
session.cookie_domain	no value	no value
session.cookie_httponly	Off	Off
session.cookie_lifetime	0	0
session.cookie_path	/	/
session.cookie_secure	Off	Off
session.entropy_file	/dev/urandom	/dev/urandom
session.entropy_length	32	32
session.gc_divisor	1000	1000
session.gc_maxlifetime	1440	1440
session.gc_probability	0	0
session.hash_bits_per_character	5	5
session.hash_function	0	0
session.lazy_write	On	On
session.name	PHPSESSID	PHPSESSID
session.referer_check	no value	no value
session.save_handler	files	files
session.save_path	/var/lib/php/sessions	/var/lib/php/sessions
session.serialize_handler	php	php

Figure 17: phpinfo showing session.cookie_httponly and session.cookie_secure both disabled

4. Risk Analysis

The table below summarizes the severity of each vulnerability based on the potential impact and the CVSS v3.1 scoring standard.

Vulnerability	Severity	CVSS Score	Impact Summary
SQL Injection	Critical	9.8/10	Full database access, data theft, authentication bypass, potential server compromise.
XSS	Medium	6.1/10	Session hijacking, credential theft, malicious redirects to attacker-controlled pages.
IDOR	Medium	6.5/10	Unauthorized access to any user record without authentication.
Authentication Issues	Critical	9.8/10	Account takeover through unlimited automated brute force attempts.
Misconfiguration	Medium	5.3/10	Server version disclosure, cookie theft via JavaScript, clickjacking exposure

I have used the CVSS v3.1 standard when referencing the scores https://www.first.org/cvss/.

4.1 Real World Attack Scenarios

SQL Injection If this was a real shopping site or any platform with a login, an attacker could pull the whole user table out of the database such as emails, usernames, password hashes, everything. Tools like 'sqlmap' can automate this in minutes with no real technical skill needed. If the database account also has write access, it gets worse as they can drop files on the server and run commands remotely.

XSS (Reflected) The most common use of reflected XSS is session hijacking. An attacker creates a link with the script in it and sends it via phishing. The victim clicks it while logged in, the script runs silently, and their session cookie gets sent to the attacker. At that point the attacker is logged in as them as it doesn't need a password.

IDOR This one is simpler to exploit than most people expect. You just change a number in the URL and you get someone else's data. In a medical records system or a banking app that would be a serious breach. There have been real cases of these affecting millions of users and the attacker just had to increment an ID without having any special tools or no hacking knowledge.

Authentication Issues Without any lockout or rate limiting, an attacker can just automate login attempts. They would use a leaked password list from a previous breach and try each one. Passwords like 'password' and 'admin123' come up more often than we expect. The application would never stop them.

Security Misconfiguration The 'phpinfo.php' file is the main issue here. It tells anyone who visits it exactly what PHP version is running as in this case 7.0.30, which hasn't had security patches

since 2019. An attacker just searches for CVEs for that version and has a list of working exploits ready. Combined with the missing 'HttpOnly' flag and the XSS vulnerability, stealing the admin session cookie becomes a straightforward chain.

5. Mitigation

Below are a few recommended fixes for each vulnerability which I found. Code-level examples are also included where applicable.

SQL Injection: The main thing that needs to change here is how the application handles user input before it reaches the database. Instead of putting the input directly into the query, prepared statements should be used. So this way the database always treats the input as data and never as part of the query itself. On top of that, the application should check that the input is actually a number before doing anything with it, since the User ID field should never accept anything else.

XSS: The issue is that whatever the user types gets printed straight into the page without any checking. Before displaying user input, it should be encoded so that characters like < and > are converted to their safe equivalents. This stops the browser from treating the input as HTML or JavaScript. Adding a Content-Security-Policy header would also help limit what scripts are allowed to run on the page.

IDOR The application needs to check not just whether a record exists, but whether the person requesting it is actually allowed to see it. Right now, it returns whatever ID you ask for with no questions asked. A simple ownership check in the query would fix this. It would also help to use random IDs rather than sequential numbers so that other records cannot just be guessed.

Auth Issues The login form needs some basic protection added. After a few wrong attempts the account should be locked for a period of time, and there should be a delay or CAPTCHA to slow down automated tools. Sending credentials as GET parameters in the URL is also a problem and POST should be used instead so they are not visible in the browser history or server logs.

Misconfiguration The 'phpinfo.php' file should just be deleted as there is no reason for it to be publicly accessible on any real server. The missing security headers are easy to add through the Apache configuration and would immediately close several attack vectors. The cookie settings in php.ini also need to be updated so session cookies are protected from JavaScript access and only sent over HTTPS.

Conclusion

Five vulnerabilities were identified and demonstrated across the DVWA application including SQL Injection, XSS, IDOR, missing brute force protection, and security misconfiguration. Each one has been documented with evidence and a recommended fix.

SQL Injection was the most serious finding. Full database access was possible with a basic payload and no special tools. The misconfiguration issues made things worse as an outdated PHP version exposed through 'phpinfo.php', missing security headers, and unprotected session cookies all add up. If someone wanted to chain these together, the path from XSS to stolen admin session is not complicated.

None of the fixes are particularly difficult. Prepared statements, encoding output, checking who is actually allowed to access a record, adding a lockout on the login form, and cleaning up the server configuration would address everything in this report. Most of it comes down to not trusting user input and not leaving default settings in place.